

Power Grid Vulnerability Analysis Using Adversarial Reinforcement Learning

Valentin Belardi, Mathéo Taillandier, Yasmine Tligui, Amine Mrabet, Ismail Filali
Supervisors: Ismail Nejjar, Volkan Cevher

Abstract—We address the challenge of identifying critical power grid vulnerabilities by framing it as a reinforcement learning problem, using a GNN-based agent trained via PPO. Unlike traditional combinatorial methods, our approach efficiently learns to prioritize line disconnections through interaction with a power flow simulator, showing competitive performance on standard test cases.

Index Terms—Reinforcement learning, Power Grid, PPO, GCN

I. INTRODUCTION

ENSURING the resilience of power grids is essential for both national security and economic continuity, as underscored by events such as the widespread blackout in Spain. Such events highlight the critical importance of developing reliable methods to assess and reinforce grid stability. Traditional methods for assessing grid vulnerability often rely on exhaustive combinatorial searches, which become computationally infeasible for real-world networks. For instance, evaluating all possible combinations of just three simultaneous line failures in a 300-line network entails analyzing over 4.5 million scenarios. To address this scalability challenge, we propose a novel framework that integrates Reinforcement Learning (RL) with Graph Neural Networks (GNNs). This approach enables efficient identification and prioritization of high-risk failure combinations across varied and complex grid topologies, offering a more practical and intelligent solution for proactive risk mitigation and grid protection.

II. RELATED PAST WORK

Reinforcement and machine learning have become valuable tools for addressing the challenges of analyzing and securing power grids. Traditional methods often struggle with the complexity and scale of modern electrical networks, prompting the exploration of data-driven techniques that can better model and predict grid vulnerabilities. Several studies have contributed to this effort, and we discuss key works that have shaped our approach.

In the context of adversarial attacks on Graph Neural Networks (GNNs), prior research has formulated the problem as a Markov Decision Process (MDP), where an attacker agent learns to inject harmful nodes into the graph to degrade performance. This is done using Proximal Policy Optimization (PPO), which allows the agent to learn effective attack strategies. Techniques like APPNP and importance sampling further enhance stability during training [1].

However, these methods often assume full access to the graph and the model, limiting their relevance in black-box or physically constrained environments.

Another important direction focuses on modeling cascading failures in power systems using RL. For example, [2] describes an MDP where the agent disconnects power lines and receives rewards based on the number of resulting outages. They explore dynamic programming techniques for small grids and Monte Carlo methods for larger ones. Yet, these approaches rely on simplified conditions, such as fixed demand profiles and full observability, which may not hold in real-world grids.

These works highlight both the strengths and limits of RL-based methods in addressing security threats in graph-structured or physical networks. Many focus on abstract or idealized settings without accounting for practical constraints.

To overcome these gaps, our work integrates Graph Neural Networks (GNNs) with Proximal Policy Optimization (PPO) to detect and exploit critical vulnerabilities in realistic power grids. By learning attack policies over graph-based representations and incorporating a physically meaningful reward signal, our method offers a scalable solution for evaluating cascading failure risks under realistic operating conditions.

III. METHOD

A. Global Approach Scheme

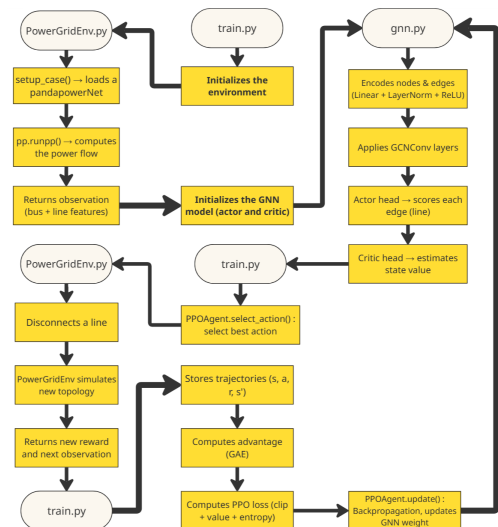


Fig. 1. Detailed project workflow

B. N - k Contingency Analysis

In power system security analysis, an N - k contingency analysis refers to the systematic study of grid stability under simultaneous outages of k or fewer system components, typically transmission lines. Given a power network with N lines, the goal is to assess the impact of all possible combinations of up to k line removals and identify those combinations that lead to instability, such as voltage violations, thermal overloads, islanding, or full system collapse.

Formally, we define a contingency set as a subset $\mathcal{C} \subseteq \{1, \dots, N\}$ such that $|\mathcal{C}| \leq k$, and analyze the altered grid topology resulting from removing the elements of $\mathcal{C}$. For each such configuration, an AC power flow analysis is conducted to determine whether the network can still operate within specified operational bounds. These include maintaining acceptable voltage levels at buses, not exceeding thermal ratings of remaining lines, and ensuring that all loads are still served by connected generation.

N - k contingency analysis plays a critical role in both planning and real-time operations of power systems. It helps grid operators proactively identify vulnerabilities, improve network robustness, and comply with reliability standards such as those set by NERC (North American Electric Reliability Corporation), which typically require networks to remain stable under any N -1 (and often N -2) contingency. However, for larger values of k or larger systems, the combinatorial explosion of possible outage scenarios makes exhaustive analysis computationally infeasible, motivating the use of scalable learning-based alternatives.

C. Simulation Environment

1) *Pandapower*: To simulate realistic power grid behavior, we adopt Pandapower, an open-source Python-based power system analysis tool designed for steady-state power flow calculations. Pandapower provides a comprehensive framework for modeling the physical structure and electrical properties of transmission networks, including buses, lines, loads, and generators. It supports the AC power flow formulation based on the Newton-Raphson method and allows the user to compute voltage magnitudes, phase angles, and line loadings in a consistent way.

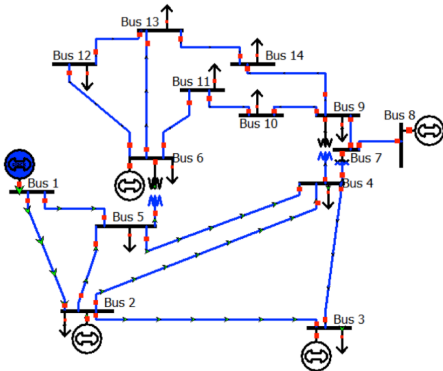


Fig. 2. IEEE 14-bus system

In practice, we initialize each scenario using predefined test cases such as case30 or case118, which are loaded into a pandapowerNet object. The base power flow is computed once at the start of each episode, and subsequent power flow solutions are recomputed whenever the topology is modified (e.g., after line disconnections). From each updated solution, we extract relevant bus features, such as voltage magnitudes, phase angles, and power injections, as well as line features including impedance, thermal capacity, and current loading. These values form the observation space of the environment, which the learning agent uses to make decisions.

This modeling approach enables the simulation of physical constraints and operating conditions in a realistic and dynamically responsive environment, which is essential for learning-based approaches to contingency analysis.

2) *Attacker agent*: We wrap the Pandapower network into a custom Gym environment in "PowerGridEnv.py" so that an "attacker" agent can sequentially choose lines to cut. The action space is 'Discrete(n_lines)', and an action mask prevents reselection of already removed lines. On each 'step()', the chosen line is disabled, 'runpp()' is re-invoked, and the environment reports whether the grid collapsed or islanded, plus a continuous penalty proportional to squared voltage and loading violations. This formulation turns contingency search into an episodic Markov decision process.

D. Features extraction

To accurately represent the electrical state of the power grid, multiple types of features are extracted from the PowerGridEnv environment using data from pandapower. These features describe the properties of buses, transmission lines, generators, as well as the network's topological structure. Each feature set is crucial for correctly modeling the physical and topological aspects of the system.

- **Bus features:** For each bus i , a 4-dimensional feature vector is collected, forming a matrix $\mathbf{B} \in \mathbb{R}^{N_b \times 4}$, where N_b is the total number of buses, describing by the following characteristics:
 - V_i = voltage magnitude at the bus in per unit, extracted from `res_bus.vm_pu`,
 - θ_i = voltage phase angle in degrees, extracted from `res_bus.va_degree`,
 - P_i = active power injection at the bus in megawatts (MW), from `res_bus.p_mw`,
 - Q_i = reactive power injection at the bus in megavars (MVar), from `res_bus.q_mvar`.
- **Line features:** For each transmission line l , a 5-dimensional feature vector is constructed, forming $\mathbf{L} \in \mathbb{R}^{N_l \times 5}$, where N_l is the total number of lines, described by:
 - ℓ_l = line length in kilometers, from `line.length_km`,
 - r_l = resistance per kilometer (ohms/km), from `line.r_ohm_per_km`,

- x_l = reactance per kilometer (ohms/km), from `line.x_ohm_per_km`,
- `max_load` = a constant fixed by the environment representing the maximum normalized load allowed on the lines,
- ρ_l = current loading percentage of the line, from `res_line.loading_percent`.
- **Generator features:** In addition to features describing the internal network elements, it is important to include features from external sources connected to the grid. We have access to key characteristics of both generators and external grids, including the following:
 - P_g = active power generated in MW (`gen.p_mw` for conventional generators, `res_ext_grid.p_mw` for external grids),
 - V_g = voltage magnitude generated in per unit (`gen.vm_pu` or `ext_grid.vm_pu`).
- **Topology features:** The structure of the network is represented by an integer matrix with two columns and one row per transmission line. Each row corresponds to a single line and contains the indices of the two buses it connects: the “from bus” and the “to bus”. These bus indices specify how nodes are linked in the grid and are extracted from the network’s `line.from_bus` and `line.to_bus` attributes.

Together, these features form a comprehensive observation of the current network state. The environment provides several types of features describing the power grid state: bus features, line features, generator features, topology, and an action mask indicating which lines can be operated on. Among these, the GNN specifically uses the bus features as node attributes and the line features as edge attributes to build its graph representation.

The topology feature, which encodes the connectivity between buses as pairs of indices, defines the graph structure and is represented as an edge index tensor listing source and target nodes for each edge.

E. Graph Neural Network

Power grids naturally lend themselves to graph-based representations, where buses are modeled as nodes $\mathcal{V}$ and transmission lines as edges $\mathcal{E}$, forming a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i$ (e.g., voltage magnitude, phase angle, active/reactive power), and each edge $(i, j) \in \mathcal{E}$ carries a feature vector $\mathbf{e}_{ij}$ (e.g., resistance, reactance, thermal limit, current loading). To leverage this structure, we employ a Graph Neural Network (GNN) to embed the electrical state of the grid into a latent space that captures both topological and physical information relevant for decision-making.

A typical GNN operates by updating node representations through local neighborhood aggregation. Formally, at layer $l +$

1, the embedding of node i is computed as:

$$\mathbf{h}_i^{(l+1)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} f_{\text{agg}}(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}, \mathbf{e}_{ij}) \right), \quad (1)$$

where $\mathbf{h}_i^{(l)}$ is the hidden state of node i at layer l , $\mathcal{N}(i)$ denotes the set of neighboring nodes, f_{agg} is an aggregation function, and $\sigma(\cdot)$ is a non-linear activation function such as ReLU.

In our implementation, we adopt the Graph Convolutional Network (GCN) model proposed by Kipf and Welling, which simplifies the aggregation by assuming symmetric adjacency and shared weights. The update rule is given by:

$$\mathbf{H}^{(l+1)} = \sigma \left(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right), \quad (2)$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency matrix with self-loops, $\tilde{\mathbf{D}}$ is the corresponding degree matrix, $\mathbf{W}^{(l)}$ is the learnable weight matrix at layer l , and $\mathbf{H}^{(l)}$ is the matrix of node embeddings.

To evaluate each candidate line removal action, we extract the embeddings $\mathbf{h}_i$ and $\mathbf{h}_j$ of the two terminal nodes of edge (i, j) , and concatenate them with the transformed edge feature vector $\mathbf{e}'_{ij}$, obtained via a linear encoder. This results in:

$$\mathbf{z}_{ij} = [\mathbf{h}_i \| \mathbf{h}_j \| \mathbf{e}'_{ij}], \quad (3)$$

where $\|$ denotes vector concatenation. The resulting vector $\mathbf{z}_{ij}$ is then passed through an actor head, implemented as a two-layer multilayer perceptron (MLP), to compute a scalar logit score for selecting edge (i, j) :

$$s_{ij} = \text{MLP}_{\text{actor}}(\mathbf{z}_{ij}). \quad (4)$$

Simultaneously, a critic head aggregates global information by applying a mean pooling over all node embeddings, and estimates the scalar value of the current state:

$$V(s) = \text{MLP}_{\text{critic}} \left(\frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \mathbf{h}_i \right). \quad (5)$$

This GNN-based actor-critic architecture enables the agent to make topology-aware decisions by leveraging both local edge context and global grid structure.

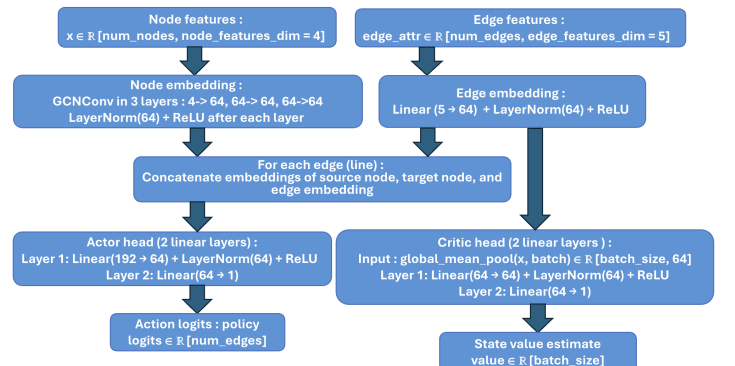


Fig. 3. GNN architecture

1) *Alternative architectures and limitations:* Other GNN variants such as Graph Attention Networks (GAT) or GraphSAGE have been considered. GATs, for instance, introduce attention mechanisms that dynamically weigh the importance of neighbors, which may improve performance on non-uniformly connected grids. However, we selected GCNConv for its computational simplicity and empirical effectiveness in static graph environments. A limitation of this approach is its reliance on local graph structure and stationarity assumptions, which may affect generalization when applied to topologies not seen during training.

F. Policy: Proximal Policy Optimization

To train the agent to identify vulnerable lines in the power grid, we use *Proximal Policy Optimization* (PPO), a widely adopted on-policy reinforcement learning algorithm known for its balance between performance and stability. PPO belongs to the class of policy gradient methods, where the objective is to maximize the expected return by updating the policy parameters θ in the direction of the gradient:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_t \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \hat{A}_t \right], \quad (6)$$

where $\hat{A}_t$ is an estimator of the advantage function at time t .

Instead of applying large policy updates, PPO constrains updates by clipping the objective function to avoid performance collapse. The clipped surrogate objective is defined as:

$$\mathcal{L}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (7)$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ is the probability ratio and ϵ is a hyperparameter.

Our implementation follows an *actor-critic* architecture, where the actor selects actions (i.e., line removals) and the critic estimates the value function $V(s_t)$ to guide learning. At each iteration, the agent collects a batch of trajectories $\{(s_t, a_t, r_t, \log \pi_{\theta}(a_t | s_t), V(s_t))\}$, from which the advantage is computed using *Generalized Advantage Estimation* (GAE):

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \text{with } \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad (8)$$

where γ is the discount factor and λ trades off bias and variance.

Because the environment state is embedded in a graph, trajectories are converted into batches of graph-structured data, including node features, edge features, and action masks. The actor network, implemented as a Graph Neural Network (see Section III-E), assigns a logit score s_{ij} to each removable line (i, j) , which is then transformed into a stochastic policy via a categorical distribution. The critic network shares the same GNN encoder and uses global mean pooling to estimate the scalar value of each graph state.

The total loss function minimized during training combines three components:

$$\mathcal{L}_{\text{total}} = \mathcal{L}^{\text{PPO}} + c_v \cdot \mathcal{L}_{\text{value}} + c_e \cdot \mathcal{L}_{\text{entropy}}, \quad (9)$$

where $\mathcal{L}_{\text{value}} = (V(s_t) - \hat{R}_t)^2$ is the value function loss, $\mathcal{L}_{\text{entropy}} = -\mathbb{E}_t[\mathcal{H}(\pi_{\theta}(\cdot | s_t))]$ encourages exploration, and c_v, c_e are weighting coefficients.

This setup, implemented in `Train.py`, leverages the strengths of PPO in graph-structured, constrained environments and enables the agent to learn scalable, topology-aware attack strategies through stable iterative updates.

G. Reward Computation in the Environment

At each step, the environment computes an immediate reward r_t reflecting the impact of disconnecting a transmission line on the power grid. The reward function distinguishes between three distinct situations:

a) *Collapse:* If the power flow calculation fails to converge following the disconnection, indicating a total grid failure or blackout, the environment returns a large fixed reward denoted $R_{\text{collapse}} = 50.0$. This strongly incentivizes the agent to cause such catastrophic failures.

b) *Islanding:* If the grid remains operational but fragments into isolated subnetworks (or islands), the environment detects this via the presence of invalid or undefined voltage values (NaNs) at some buses. This situation represents a partial but significant disruption of the grid. The agent receives a moderate reward equal to half the collapse reward, $R_{\text{islanding}} = 25.0$.

c) *Operational deviations:* When the grid remains stable and connected, the reward quantifies stress on the network through voltage and line loading deviations:

- **Voltage violations:** For each bus, the squared normalized deviation of its voltage magnitude outside the acceptable range $[v_{\min} = 0.95, v_{\max} = 1.05]$ is computed and averaged across all buses, yielding J_V .
- **Line overloads:** For each transmission line, the squared overload beyond the maximum loading threshold ($\ell_{\max} = 1.0$, meaning 100% loading) is calculated and averaged, yielding J_L .

These components are combined with a weight favoring line violations more heavily:

$$\text{total_violation} = J_V + 1.5 \times J_L \quad (10)$$

To capture the geographical dispersion of violations, a spread factor $S \geq 1$ is computed based on the average physical distance between buses experiencing violations. This factor amplifies the reward if failures are spread widely across the network:

$$S = 1.0 + \frac{\text{mean distance between violated buses}}{100} \quad (11)$$

The immediate reward in this stable but stressed case is then:

$$r_t = \text{total_violation} \cdot S \quad (12)$$

d) *Penalty for successive disconnections:* To incentivize the agent to cause significant disruption using as few line disconnections as possible, the immediate reward, even if it's a collapse reward, island reward or deviation reward, is multiplied by a factor that decreases exponentially with the number of lines already disconnected, denoted n_t , leading to our final reward:

$$r_t \leftarrow r_t \times 0.9^{n_t-1} \quad (13)$$

The designed reward structure guides the agent to effectively destabilize the power grid by prioritizing actions that lead to critical failures such as collapse or islanding, while also accounting for operational stresses in stable conditions. By incorporating a spatial spread factor and penalizing excessive line removals, the agent is encouraged to discover the most impactful attack strategies that maximize disruption efficiently and strategically.

H. Training Strategy

Episodes: The agent interacts with the environment in episodes. Each episode consists of the agent removing a set number of lines from the grid to identify vulnerabilities. The episode ends when either a severe grid failure (collapse) occurs or when the maximum number of line removals ($k = 5$) is reached.

Number of Steps: Training is performed over a total of 30,000 steps, with updates occurring every 256 steps. This means the agent collects data for 256 steps before updating its policy.

Action Masking: An action mask is used to ensure that the agent does not choose lines that have already been removed.

State Management: After selecting an action, the environment's state is updated. The removed line is marked as out of service, and the grid's stability is recalculated using power flow analysis. The state consists of grid features, topology, and the internal state of removed lines.

Policy and Optimization: Now that we have the consequences from the last action, PPO intervenes to update the agent's policy. It adjusts the policy to increase the likelihood of selecting actions that lead to higher rewards in the future. The logits generated by the GNN guide this process by representing the likelihood of each action leading to a critical vulnerability. These logits are then transformed into a probability distribution, from which the agent samples its next action. The agent will explore actions with lower probability when uncertain, allowing for broader exploration of the state space, and exploit the actions with higher probability based on the learned model, ultimately guiding the agent toward an optimal solution over time.

Reward Calculation: At each network state update, the reward is calculated based on the network stability after the action, taking into account voltage violations, line load violations, and collapse or islanding cases, as shown before.

That's the different actions that include our training loop. We fixed the maximum number of lines the agent is allowed to remove in each episode to 5, ensuring that the agent focuses on the most critical vulnerabilities.

The agent's performance is evaluated by comparing the model-based search (DPO) with an exhaustive search. The exhaustive search evaluates all possible line removals in the network, testing every combination. This method serves as a benchmark to ensure that the DPO models that we trained are able to capture almost the full range of vulnerabilities.

The model's performance is compared to the exhaustive search by:

- **Matching Results:** Identifying overlapping vulnerabilities between the two methods.
- **Ranking and Scoring:** The vulnerabilities are ranked by their individual reward scores, with the most critical combinations being prioritized. After ranking the vulnerability combinations based on their rewards, each model selects the most critical ones, sums their respective rewards, and computes a final score. This final score is then used to compare the performance of the DPO and exhaustive search models.
- **Model Efficiency:** Assessing how quickly the model identifies vulnerabilities compared to exhaustive search.

To training, we used the following cases from the power grid network library: case14, case30, case39, case57, case300. The challenge: selecting networks for which performing the exhaustive search with all combinations is feasible and not too computationally expensive. We also had another challenge, since we are training on smaller networks, the model needs to be capable of generalizing well to larger cases. This is where the testing phase comes into play, allowing us to evaluate how well the model can adapt to larger, more complex networks.

I. Testing Phase

During testing, the trained GNN policy, obtained by loading the weights learned via PPO during training, is fixed and evaluated on the same power grid environment but with different cases. The number of allowed line removals is reduced to $k = 3$ to limit test complexity and allow meaningful comparisons across cases. Unlike training, the policy is no longer updated; the objective is to evaluate the model's generalization capability.

We implemented two evaluation modes:

- **Stochastic Mode:** The agent samples actions from the probability distribution defined by the GNN's output logits (after masking). This introduces variability and tests the agent's robustness across different paths. Although implemented, this mode was not used for final evaluation.

- **Deterministic Mode:** At each step, the agent selects the action with the highest probability based on the logits computed by the GNN, strictly following the most confident prediction. We use this mode during testing to assess the agent’s optimal behavior in a controlled and reproducible setting, without the randomness introduced by exploration.

The key metrics collected during evaluation include total rewards, the number of steps per episode, success rate (the frequency of grid collapse), average bus voltage violation ratio, and line loading violation ratio. These metrics help to evaluate the overall performance and stability of the learned policy in the test environment.

To assess whether the trained model can detect the most critical vulnerabilities in a test power grid, we perform a guided vulnerability analysis. Instead of exhaustively testing all possible combinations of line removals (which becomes intractable on large grids), we limit the search to 5^k paths per test case. At each step of the tree search (up to depth $k = 3$), the model uses the logits from the GNN to rank all lines and select the top 5 most vulnerable candidates for removal. These are recursively combined to build up to $5^3 = 125$ sequences of line removals.

Each of these sequences is simulated in a cloned environment to avoid side effects. For every sequence, we compute a vulnerability score using the same reward logic as in training (collapse > islanding > violations). At the end, the most impactful sequences are then ranked and printed, showing the removed lines, their associated score, and whether they caused a collapse, islanding, or stress. This ranked output gives a concise summary of the failure modes the model believes to be most dangerous, providing insight into how well the agent internalized the grid’s structural weaknesses.

To improve efficiency in the tree search, we implement pruning: if removing a given subset of lines already causes the grid to collapse (or island), we skip all longer sequences that contain this subset, since their impact is already captured. This allows us to avoid redundant evaluations and focus only on novel and meaningful vulnerabilities.

Unlike the deterministic test phase, where the agent follows a single optimal policy trajectory per episode, this tree-based approach reveals whether the model consistently ranks and combines critical vulnerabilities across multiple branches. It adds a complementary layer of evaluation, highlighting the model’s ability to uncover high-impact failure points beyond its primary trajectory.

IV. RESULTS

A. Generalization capability

To evaluate the effectiveness of our GNN-based agent, we compare it against exhaustive search in terms of computational cost and quality of the top-5 vulnerability sets discovered. The two charts below highlight the scalability and accuracy trade-offs between both approaches.

Scoring system: 50 if collapsed, 25 if islanded, penalty applied for each line removed.

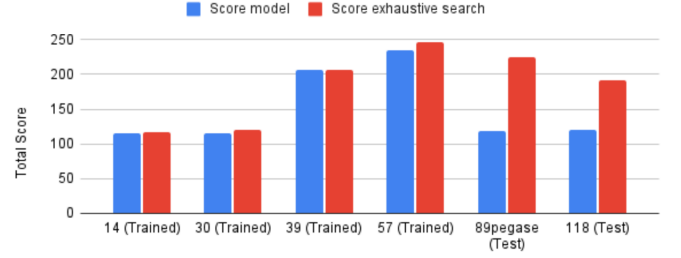


Fig. 4. Score Sum of Top 5 Vulnerabilities: Model-based vs Exhaustive search

The first plot compares the total score of the top-5 most damaging line removal combinations discovered by each method, using a scoring system that prioritizes collapses (50 points), islanding (25 points), and penalizes multi-line attacks. On trained grids, the model closely matches the exhaustive search in total score, validating that it learns to prioritize impactful configurations. On unseen test cases (89pegase, 118), the model achieves reasonable performance despite never having seen those grids during training, suggesting a degree of generalization in its learned policy.

For larger cases, exhaustive search was not feasible, but our model successfully produced the following cumulative scores for the top 5 vulnerabilities:

- Case145: 117.5
- Case1354pegase: 102.5
- Case1888rte: 115

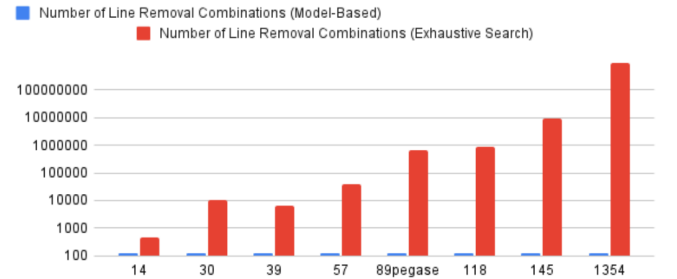


Fig. 5. Comparison of computational costs between Model-based and Exhaustive search methods

Figure 5 presents a logarithmic-scale comparison of the number of line-removal combinations evaluated by the model-based method versus exhaustive search across multiple benchmark power grids. In the exhaustive setting, for a power grid with L transmission lines, the total number of possible line-outage combinations of size up to k is given by:

$$\sum_{i=1}^k \binom{L}{i} \quad (14)$$

This combinatorial growth rapidly becomes infeasible as L increases. For example, if the grid contains $L = 1991$ lines (as in the PEGASE-1354 case), evaluating all 3-line outage scenarios requires testing approximately $\binom{1991}{3} \approx 1.3 \times 10^9$ combinations. In contrast, the model-based agent selects a single action per step and operates under a fixed number of

environment interactions per episode, resulting in a constant-time complexity per decision. This sharp contrast in computational cost highlights the scalability advantage of our learning-based method for large-scale contingency analysis tasks, where exhaustive enumeration is prohibitive.

B. Performance increase

To assess the convergence behavior and learning dynamics of the agent, we track key performance indicators over time—namely the average episodic reward, policy loss, and policy entropy—across three distinct phases of training.

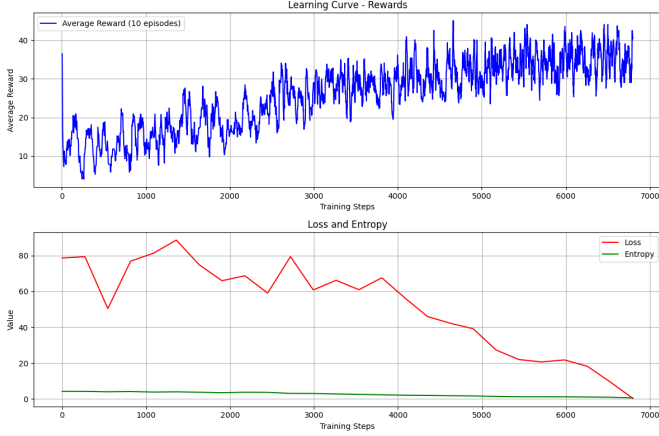


Fig. 6. Reward, entropy and loss curves over time

In the early stage of training, the agent exhibits high reward variance with a slow upward trend, indicating initial learning progress. The PPO loss fluctuates significantly across batches, reflecting unstable policy updates, which is common in the early exploration phase. Entropy remains mostly flat, suggesting consistent exploration without premature convergence.

As training continues, the average reward increases more steadily, demonstrating that the agent is learning to identify more impactful actions. The loss curve shows a general downward trend with occasional spikes, typical of PPO's clipped objective which allows some oscillations while maintaining stability. Entropy gradually decreases, suggesting that the agent's policy is becoming more focused and confident.

In the final phase, the agent reaches higher average rewards with reduced volatility, indicating convergence to an effective policy. The PPO loss steadily declines, reflecting successful optimization of both actor and critic networks. Meanwhile, entropy approaches zero, showing that the agent has shifted toward deterministic decisions after sufficient exploration.

Notably, the mean reward improves from around 10 (typically associated with no significant grid disruption) to approximately 40, reflecting the agent's increasing success in inducing collapse or islanding events through targeted line removals.

V. CONCLUSION

In this work, we proposed a reinforcement learning framework for identifying vulnerable line combinations in power grids using a graph-based actor-critic architecture. By embedding the physical and topological structure of the grid into a Graph Neural Network and training the agent with Proximal Policy Optimization, our approach efficiently learns to prioritize critical contingencies while significantly reducing the computational burden compared to exhaustive search. Experimental results on standard test cases demonstrate that the model achieves competitive performance in ranking failure scenarios while maintaining scalability across increasingly large systems.

REFERENCES

- [1] D. Gupta, R. Mudumbai, T. Saha, M. Behl, J. Pomeranz, V. Venkatasubramanian, and A. D. Domínguez-García, "A deep reinforcement learning framework for real-time control and resilience of power distribution networks," *Neurocomputing*, vol. 545, p. 127260, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231223008305>
- [2] J. M. Sunyer Nestares, "Power system security analysis using reinforcement learning," Master's thesis, University of Illinois at Urbana-Champaign, 2019. [Online]. Available: <https://www.ideals.illinois.edu/items/115356>